Exploratory Data Analysis in Python

IndabaX Zimbabwe | Deep Learning Community

Data Cleaning · Validation · Analysis · Visualization

Walter M Nyamutamba: Technical & Community Impact Lead

What is Exploratory Data Analysis?

EDA is the process of **reviewing and cleaning data** to derive insights and generate hypotheses.

Derive Insights

Uncover hidden patterns, relationships, and trends in your dataset.

Generate Hypotheses

Form testable questions to guide further modelling and investigation.

Clean the Data

Detect and fix missing values, wrong data types, and outliers.

Why is Missing Data a Problem?

Affects Distributions

Missing heights of taller students shifts the sample mean away from the true population mean your statistics become unreliable.

Less Representative

Certain groups (e.g. oldest or highest-paid workers) are disproportionately absent, creating bias in your findings.

Incorrect Conclusions

Analysing incomplete data can lead to flawed insights and poor real-world decisions, undermining trust in your work.

Data Professionals' Job Dataset

Dataset used throughout this workshop — 8 columns, 600 rows:

Column	Description	Data Type
Working_Year	Year the data was obtained	Float
Designation	Job title	String
Experience	Level e.g. "Mid", "Senior"	String
Employment_Status	Type e.g. "FT", "PT"	String
Employee_Location	Country of employment	String
Company_Size	Size label e.g. "S", "M", "L"	String
Remote_Working_Ratio	% of time working remotely	Integer
Salary_USD	Salary in US dollars	Float

Checking for Missing Values

```
print(salaries.isna().sum())
```

```
Working_Year              12
Designation               27
Experience                33
Employment_Status         31
Employee_Location         28
Company_Size              40
Remote_Working_Ratio      24
Salary_USD                60
```

Key Insights

- Use .isna().sum() to count missing values per column
- Salary_USD has the most gaps (60) — needs careful imputation
- Company_Size is second highest (40)
- Columns with ≤5% missing can be safely dropped
- Heavier columns require statistical imputation instead

Strategies for Addressing Missing Data

01 Drop Missing Values

- Use when ≤5% missing
- threshold = len(df)*0.05
- dropna(subset=cols_to_drop)

```
threshold = len(salaries)*0.05
salaries.dropna(
  subset=cols_to_drop,
  inplace=True)
```

02 Impute Summary Statistic

- Fill NaN with mode/median
- Works for categorical & numeric
- fillna(col.mode()[0])

```
for col in cols_missing[:-1]:
  df[col].fillna(
    df[col].mode()[0],
    inplace=True)
```

03 Impute by Sub-Group

- Different groups → diff medians
- Executive ≠ Entry salary
- groupby + transform + map

```
d = df.groupby('Experience')
  ['Salary_USD'].median()
  .to_dict()
df['Salary_USD'].fillna(
  df['Experience'].map(d))
```

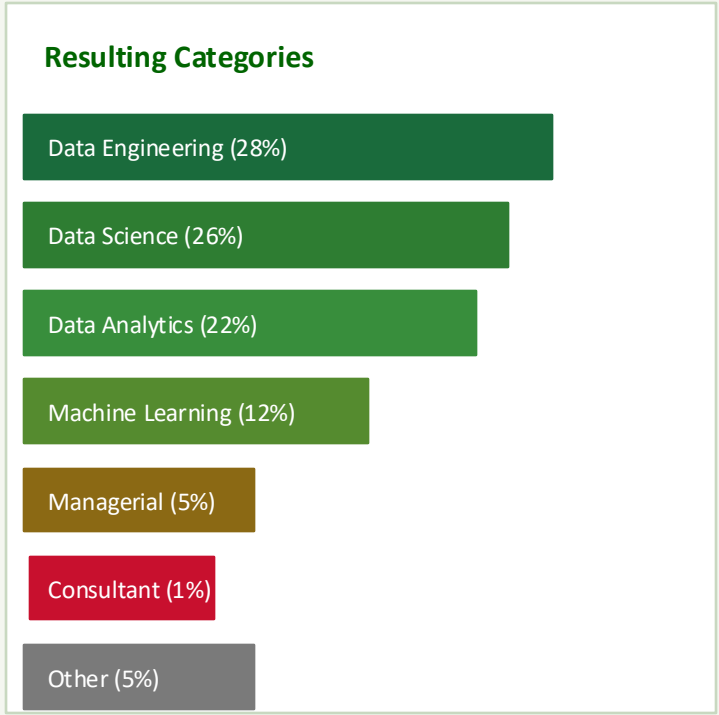
Converting & Analysing Categorical Data

Use str.contains() + np.select() to create structured category columns from free-text:

```
# Define keyword patterns
data_science = "Data Scientist|NLP"
data_analyst = "Analyst|Analytics"
ml_engineer  = "Machine Learning|AI"
manager      = "Manager|Head|Director"

# Build boolean conditions list
conditions = [
  df['Desig'].str.contains(data_science),
  df['Desig'].str.contains(data_analyst),
  df['Desig'].str.contains(ml_engineer),
  df['Desig'].str.contains(manager)]

# Create column with np.select
df['Job_Category'] = np.select(
  conditions, job_categories,
  default="Other")
```



Working with Numeric Data

1

Remove Formatting

```
df["Salary"]=df["Salary"].str.replace(",","")
```

Strip commas from 20,688,070.00 → 20688070.00

2

Convert Data Type

```
df["Salary"]=df["Salary"].astype(float)
```

Cast cleaned string column to float64

3

Create Derived Column

```
df["Salary_USD"]=df["Salary_INR"]*0.012
```

1 Indian Rupee = 0.012 USD

4

Add Group Statistics

```
df["std_dev"]=df.groupby("Experience")["Salary_USD"].transform(lambda x:x.std())
```

Attach group-level std to each row via transform()

What is an outlier

- An observation far away from other datapoints
  Median house price: $400,000

  Outlier house price: $5,000,000

  Should consider why the value is different:

  Location, number of bedrooms, overallsize etc

Handling Outliers with IQR

IQR

75th Percentile – 25th Percentile

Upper Outlier

Q75 + (1.5 × IQR)

Lower Outlier

Q25 – (1.5 × IQR)

```
# Calculate percentiles
q75 = df["Salary_USD"].quantile(0.75)
q25 = df["Salary_USD"].quantile(0.25)

# Compute IQR
iqr = q75 - q25   # 76305.0

# Thresholds
upper = q75 + (1.5*iqr)  # 251953.5
lower = q25 - (1.5*iqr)  # -53266.5

# Remove outliers
no_out = df[
  (df['Salary_USD'] > lower) &
  (df['Salary_USD'] < upper)]
```

Before vs After

Metric	With Outliers	Cleaned
Count	518	509
Mean	$104,906	$100,675
Std Dev	$62,660	$53,643
Max	$429,675	$248,257

Initial Exploration: First Steps

.head()

Preview first 5 rows — check column names, types, values

```
books.head()
```

.info()

Column dtypes, null counts, memory usage

```
books.info()
```

.describe()

Summary stats: count, mean, std, min, quartiles, max

```
books.describe()
```

.value_counts()

Frequency count of each category in a column

```
books.value_counts("genre")
```

sns.histplot()

Visualise distribution of a numerical column with histogram

```
sns.histplot(data=books, x="rating")
```

sns.boxplot()

Show median, IQR and outliers visually by group

```
sns.boxplot(data=books,x="year",y="genre")
```

Data Validation Techniques

Data Type Validation

Check with .dtypes • Fix with .astype()

String	`str`
Integer	`int`
Float	`float`
Dictionary	`dict`
List	`list`
Boolean	`bool`

Value Range Validation

Check valid genres

```
df["genre"].isin(["Fiction","Non Fiction"])
```

Invert — find invalid

```
~df["genre"].isin(["Fiction","Non Fiction"])
```

Filter to valid rows

```
df[df["genre"].isin([...])]
```

Numeric bounds

```
df["year"].min()  # 2009
df["year"].max()  # 2019
```

Boxplot by group

```
sns.boxplot(data=df,x="year",y="genre")
```

Data Summarization: groupby & agg

Aggregating Functions:

`.sum()`	`.count()`	`.min()`
`.max()`	`.var()`	`.std()`
`.mean()`	`.median()`	`.agg()`

```
# Named aggregations per group
books.groupby('genre').agg(
  mean_rating=('rating','mean'),
  std_rating =('rating','std'),
  median_year=('year','median')
)

# Visualise
sns.barplot(data=books,
  x='genre', y='rating')
```

Result:

Genre	Mean Rating	Std Dev	Med Year
Childrens	4.780	0.122	2015
Fiction	4.570	0.281	2013
Non Fiction	4.598	0.179	2013

Visualising Categories of Data

Bar Chart · Pie Chart · Box Plot · Histogram · Heatmap · Count Plot

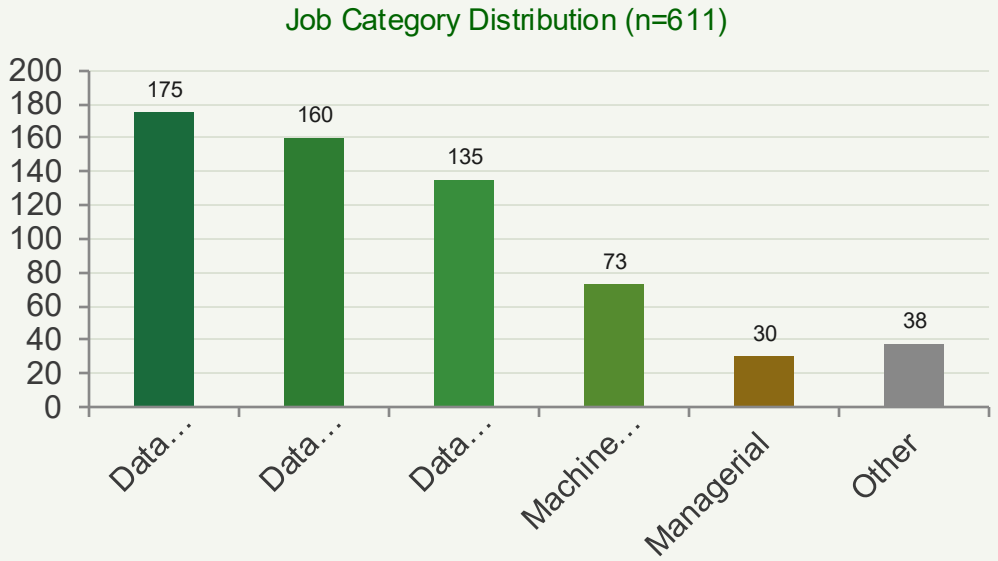
5 essential plot types with code + when to use them

Visualisation 1 of 5 — Bar Chart

Best For	When?	Avoid When
Comparing counts or means across discrete categories	Categorical X-axis, numerical Y-axis (counts, averages)	Too many categories (>15) — use horizontal bars instead

```
import seaborn as sns
import matplotlib.pyplot as plt

# Count plot (auto-counts)
sns.countplot(
  data=salaries,
  x="Job_Category",
  palette="Greens_d")
plt.title('Jobs by Category')
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()
```



Tip: Use sns.barplot() when plotting a mean/aggregate (adds confidence intervals automatically). Use sns.countplot() for raw frequency counts.

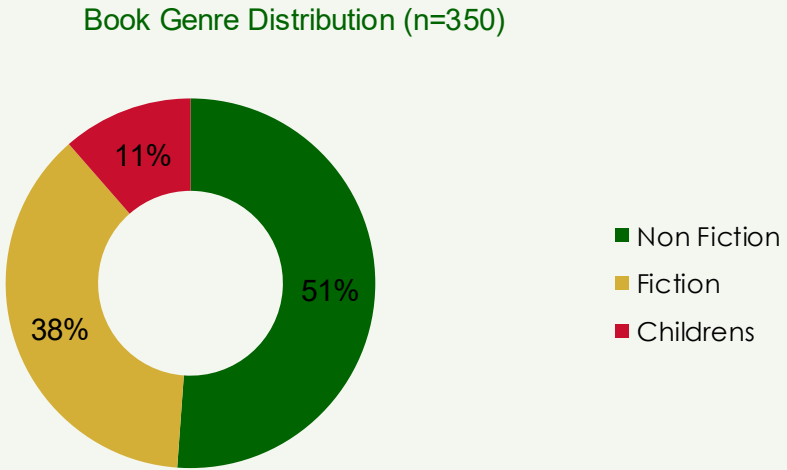
Visualisation 2 of 5 — Pie / Donut Chart

Best For	When?	Avoid When
Showing part-to-whole proportions for a small number of categories	≤5 distinct categories where relative share is the key message	Many slices or small differences — bar charts show differences more clearly

```
# Matplotlib pie chart
import matplotlib.pyplot as plt

labels = ['Non Fiction','Fiction',
          'Childrens']
sizes  = [179, 131, 40]
colors = ['#006400','#D4AF37',
          '#C8102E']

plt.pie(sizes, labels=labels,
  colors=colors,
  autopct='%1.1f%%',
  startangle=90)
```



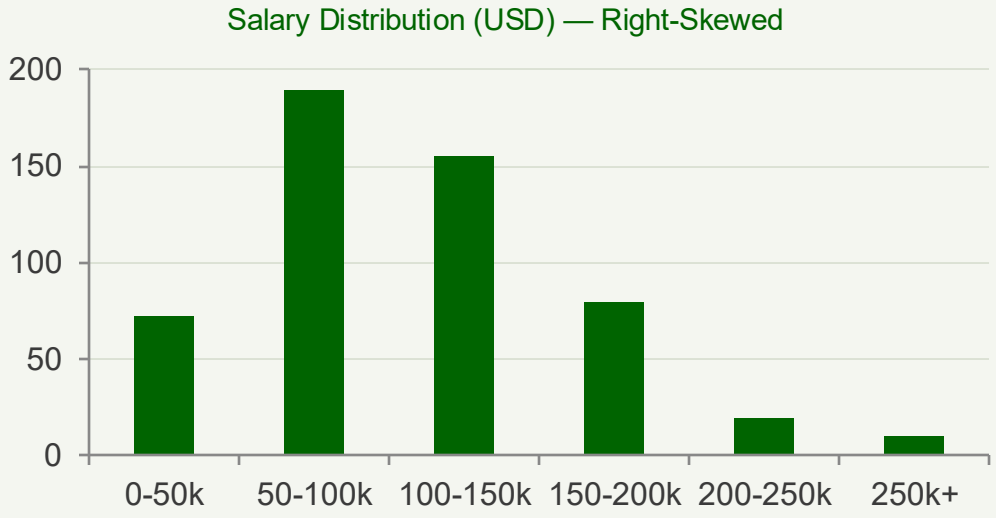
Tip: Pie charts work best with 3–5 slices. For more categories, a bar chart is easier to read. Avoid 3-D pies — they distort the actual proportions.

Visualisation 4 of 5 — Histogram

Best For	When?	Avoid When
Showing the frequency distribution of a single continuous variable	Numerical data — check for skew, gaps, bimodal peaks, or outliers	Categorical data — use countplot or bar chart instead

```
# Basic histogram
sns.histplot(
  data=salaries,
  x="Salary_USD",
  binwidth=10000,
  color="#006400",
  kde=True)

# Overlay two groups
sns.histplot(
  data=salaries,
  x="Salary_USD",
  hue="Experience",
```



Reading histograms: Right-skewed → mean > median (high-earner outliers pull average up). Use kde=True to overlay a smooth density curve for shape clarity.

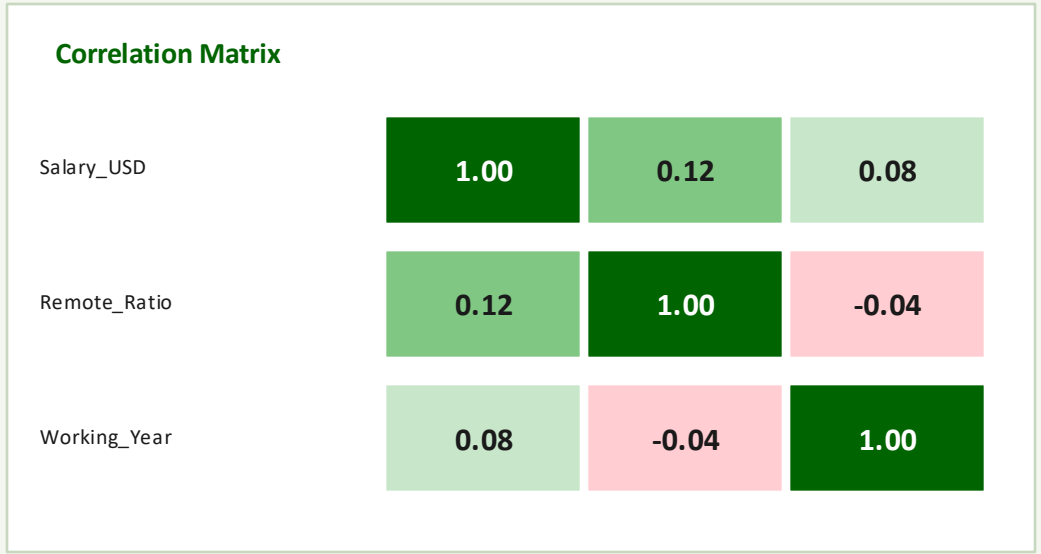
Visualisation 5 of 5 — Heatmap

Best For	When?	Avoid When
Spotting correlations or patterns between multiple numeric variables at once	Correlation matrices, cross-tabulations, pivot tables, or time × category data	Very few variables (≤3) — a scatter plot or bar chart is simpler and clearer

```
# Correlation heatmap
import seaborn as sns

corr = salaries[
  ['Salary_USD',
   'Remote_Working_Ratio',
   'Working_Year']
].corr()

sns.heatmap(corr,
  annot=True,
  cmap='Greens',
  fmt='.2f',
```



🌡 Colour scale: dark green = strong positive correlation (+1.0) → white = no correlation (0) → red = negative correlation (-1.0). Values close to 0 mean the variables are independent.

Choosing the Right Chart — Quick Reference

Chart Type	Data Shape	Use When	Seaborn / Matplotlib Code
Bar Chart	Categorical vs Numeric	Compare counts / means across categories	`sns.countplot() / sns.barplot()`
Pie / Donut	Part-to-Whole	Show proportions (≤5 groups)	`plt.pie() / sns.plot(kind='pie')`
Box Plot	Categorical + Numeric	Distribution, median & outliers per group	`sns.boxplot()`
Histogram	Single Numeric	Frequency distribution, detect skew	`sns.histplot()`
Heatmap	Numeric × Numeric grid	Correlations or pivot table patterns	`sns.heatmap()`
Scatter Plot	Two Numeric Variables	Relationship / correlation between two cols	`sns.scatterplot()`
Line Chart	Numeric over Time	Trends, changes over ordered sequence	`sns.lineplot()`

Key Takeaways

01 Always explore first — .head(), .info(), .describe() before you touch the data.

02 Handle missing values deliberately: drop ≤5%, impute by stat or sub-group but consultate the data manager first.

03 Use str.contains() + np.select() to convert free-text to structured categories.

04 Convert numeric strings with str.replace() + astype() before any calculation.

05 Detect outliers with IQR thresholds; decide keep or remove based on context.

06 Match your chart to your data: bar=categories, histogram=distribution, heatmap=correlations.

IndabaX Zimbabwe Hands on practice projects